



NARUTO

-ナルト-



火影



PROYECTO 1:
ENUNCIADO: MOSTRAR TODOS LOS REGISTROS
DE LA TABLA EQUIPO.

CONSULTA:
SELECT * FROM Equipo;
GO

Kevin Alfredo Macha Bruno

IdEquipo	Descripción
1	Equipo asignado a Dirección
2	Uso exclusivo OT
3	N/A
4	Teclado con fallas
5	Asignada a asesoría jurídica
6	N/A
7	Batería con desgaste
8	Recepción
9	Gabinete oxidado
10	Gerencia
10	Gerencia
11	No enciende, en reparación
13	Pantalla con ligera mancha
14	Imagen institucional
15	Disco lento
19	Teclado con teclas mal
18	No tiene RAM suficiente
19	Gerencia de Operaciones
20	Cámara no funciona
21	RRHH
22	Bisagra suelta
23	Poneamiento
24	Batería agotada
25	Secretaría General
26	Fuente de poder inestable
27	Atención al ciudadano
28	Placa madre dañada
29	Respaldo OT
20	Servidor de pruebas
31	Reemplazo imagen
33	Carcasa rota
33	Logística
34	Cargador dañado
36	N/A
36	Sin regulador
37	Equipo de desarrollo
38	Lector de huella no funciona
39	No arranca finanzas
41	Dirección de
42	Monitor parpadea
43	Linux para servidor local
44	Asignación interna
46	Conector de
45	Fuente quemada
47	Recursos Humanos
47	RAM insuficiente
48	Equipo de diseño gráfico
49	Reemplazo final
50	Reemplazo final



PROYECTO 1: VISUALIZACIÓN DE INVENTARIO DE EQUIPOS

1 TABLA INVOLUCRADA Y DESCRIPCIÓN

Tabla: Equipo

Contiene la información completa de los equipos de inventario (tipo, estado, observación, etc.) y sus identificadores únicos, además de referencias a dependencias y usuarios.

Basado en el esquema general SQL.

2 RELACIÓN ENTRE TABLAS (METODOLOGÍA CROWS FOOT)



3 ¿HAY INNER JOIN EN ESTA CONSULTA?

? ~~JOIN~~
NO

Es una selección simple de una sola tabla.

4 DIAGRAMA DE VENN



Un solo círculo representa el conjunto de datos completo de la tabla.

5 CONSULTA SQL

-- Proyecto 1: Mostrar todos los registros de la tabla equipo
SELECT * FROM Equipo;
GO

5 CONSULTA



7 RESUMEN DEL PROCESO

Recupera la totalidad de los datos de la tabla 'Equipo' para obtener una visión completa del inventario.

6 RESULTADO OBTENIDO

idEquipo	cód_Equipo	fecha	cód_Tecnología	tipo_equipo	estado	observacion	dependencia_id	usuario_id
1	ALM-001	2025-02-10	TEC-007	ESCRITORIO	BUENO	Teclado con tecla Enter...	5	3
2	ALM-022	2025-02-12	TEC-008	LAPTOP	BUENO	Equipo nuevo	4	6
3	ALM-003	2025-02-13	TEC-009	ESCRITORIO	BUENO	No tiene RAM suficiente	5	11
4	ALM-004	2025-02-15	TEC-010	LAPTOP	REGULAR	Gerencia de Operaciones	3	7
5	ALM-005	2025-02-20	TEC-021	ALL IN ONE	MEDIO	Cámara no funciona	2	14
6	ALM-006	2025-02-22	TEC-022	ESCRITORIO	BUENO	RRHH	1	16
7	ALM-007	2025-02-23	TEC-023	ESCRITORIO	REGULAR	Bisagra suelta	3	9
8	ALM-008	2025-02-25	TEC-024	LAPTOP	BUENO	Poneamiento	4	3

8 CONCLUSIÓN

Este primer paso es esencial para auditorías y análisis de datos, ya que proporciona una base sólida de datos completa y confiable para informes más detallados y decisiones acertadas.

Kevin Alfredo Macha Bruno

PROYECTO 2:

MOSTRAR ÚNICAMENTE EL CÓDIGO Y NOMBRE DE LOS USUARIOS



1. TABLA INVOLUCRADA Y DESCRIPCIÓN

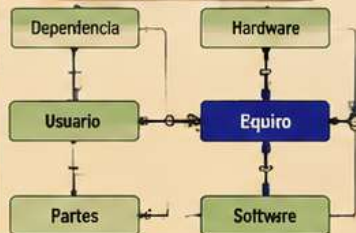


Tabla: "Equipo"

Contiene la información general de los equipos de inventario (tipo, estado, ubicación, etc.) y sus identificadores únicos, así como referencias a dependencias y usuarios.

Texto basado en el esquema general SQL.

2. RELACIÓN ENTRE TABLAS (METODOLOGÍA CROWS FOOT)



2. ¿HAY INNER JOIN EN ESTA CONSULTA?



NO

Es una selección simple de una sola tabla.

4. DIAGRAMA DE VENN



Un solo círculo representa el conjunto de datos completo de la tabla.

5. CONSULTA SQL

-- Proyecto 2: Mostrar únicamente el código y nombre de los usuarios

```
SELECT idusuario AS Código, nombreUsuario AS Nombre
FROM Usuario;
GO
```

5. CONSULTA



7. RESUMEN DEL PROCESO

Recupera la totalidad de los datos de la tabla 'Equipo' para obtener una visión completa del inventario.

6. RESULTADO OBTENIDO

idusuario	nombreUsuario
1	Luis Fernando Pérez Gómez
2	Ana Belén Torres Rojas
3	Carlos Eduardo Ramírez Soto
4	Maria Isabel Castillo Vega
5	José Antonio Morales Díaz
6	Sandra Milagros León Huamán
7	Diego Alejandro Navarro Ruiz
8	Patricia Andrea Salazar Mendoza

8. CONCLUSIÓN

Primer paso esencial para auditorías y análisis de datos, proporcionando una base de datos sólida para informes más complejos.

Código	Nombre
1	Augusto Carlos Quiñones García
2	Barrios Iván Ramírez Flores
3	Mendoza Juan Martín Herrera
4	Teje Lucía Ramírez Alvarado
5	Uribe Alberto Valverde Llano
6	Quispe Ángel Miguel Apaza
7	Apaza Sonia Marisol Cáceres
8	Lima Walter Jesús Rojas
9	Medrano Elizabeth Paola Sosa
10	Pinto Daniel Alejandro Rojas
11	Vargas Vanessa Rocío Choque
12	Gómez Oscar Raúl Condori
14	Ayala Patricio Liliana Herrera
14	Medina Héctor Pablo Calle
15	Lozano Margot Isabel Castro
16	Mamani Alejandro Mario Quispe
18	Pino César Agnoldo Valdivia
19	Contreras Cecilia Rocío Vásquez
20	Tito Rubén Darío Nina
21	Alvarado Nancy Beatriz López
22	Gutiérrez Javier Arturo Cuba
23	Jaregui Silvia Roxana Campos
24	Barriga Melissa Lila Aragón
25	Montoya Verónica Sofía Cueva
26	Rojas David Samuel Ordíñola
27	Flores Laura Jimena Ribera
28	Crespo Enrique Alonso Gutiérrez
29	Torres Fabiola Carmen Zagarra
30	Fernández Raúl Martín Apaza





UPLA



PROYECTO 3:

USO DE ALIAS DE COLUMNAS

(Nombre del Usuario) - InventarioEquipos



1. TABLA INVOLUCRADA Y DESCRIPCIÓN

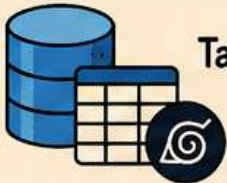
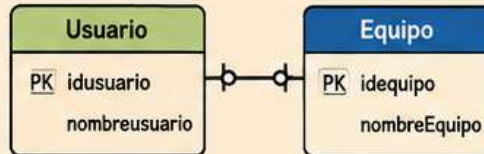


Tabla: **Usuario**

Descripción: Esta tabla almacena la información de los usuarios del sistema. Contiene un identificador único (idusuario) y el nombre completo de cada usuario.

2. RELACIÓN ENTRE TABLAS (METODOLOGÍA CROWS FOOT)



3. ¿HAY INNER JOIN EN ESTA CONSULTA?

?

~~JOIN~~

NO

Es una consulta simple sobre una sola tabla (Usuario).

4. DIAGRAMA DE VENN



Conjunto completo de datos de la tabla Usuario

nombreusuario (Todos los valores almacenados)

5. CONSULTA SQL

-- Proyecto 3: Mostrar el nombre del usuario utilizando alias personalizados.

```

SELECT nombreusuario AS [Nombre del Usuario]
FROM Usuario;

GO
  
```

6. RESULTADO OBTENIDO

	Nombre del Usuario
1	Ramirez Solano Daniel Arturo
2	Guzmán Herrera Luis Fernando
3	Mendoza Castillo Carlos Iván
4	Torres Salazar José Antonio
5	López Barrera Miguel Ángel
6	Rojas Villanueva Pedro Manuel
7	Flores Paredes Juan Esteban
8	Castro Navarro Alejandro José

7. RESUMEN DEL PROCESO



8. CONCLUSIÓN



El uso de alias (AS) permite renombrar las columnas de los resultados de una consulta, mejorando la legibilidad y comprensión de la información para el usuario final.



KEVIN ALFREDO MACHA BRUNO





PROYECTO 4:

VISUALIZACIÓN DE DATOS DE DEPENDENCIA

Enunciado: Mostrar datos de la tabla dependencia utilizando alias.



```
SELECT dep.iddependencia AS ID, dep.nombreDependencia AS Dependencia
FROM Dependencia AS dep;
GO
```

RESULTADOS

ID	Dependencia
1	Centro de Desarrollo de Software (CDS)
2	Dirección de Gestión y Finanzas
3	Talento y Desarrollo Humano
4	Logística y Abastecimiento
5	Asesoría Legal
6	Planificación y Presupuesto
7	Secretaría Administrativa
8	Operaciones y Servicios
9	Comunicación Institucional
10	Atención y Experiencia al Usuario

ESTADÍSTICAS DE CLIENTES

ESTADÍSTICAS DE CLIENTES



1. TABLA INVOLUCRADA Y DESCRIPCIÓN



Tabla: Dependencia

Almacena la información de las dependencias organizacionales de la institución.

Dependencia	
PK iddependencia	ID
nombreDependencia	Nombre de la dependencia

Descripción:

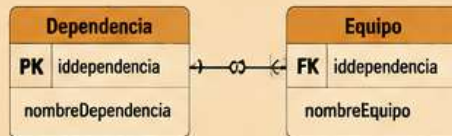
iddependencia: ID único.

nombreDependencia: Nombre de la dependencia.

NOT NULL.

2. RELACIÓN ENTRE TABLAS

(Metodología Crows Foot)



Una Dependencia puede tener asignados múltiples Equipos.

3. ¿HAY INNER JOIN EN ESTA CONSULTA?



Es una selección simple de una sola tabla (Dependencia). No se combinan datos de otras tablas.

4. DIAGRAMA DE VENN

(Representación Conceptual)



Todos los Registros de la tabla Dependencia

Columnas Seleccionadas: ID, Dependencia (con Alias)

5. CONSULTA SQL

-- Proyecto 4: Mostrar datos de la tabla dependencia utilizando alias.

```
SELECT dep.iddependencia AS ID,
dep.nombreDependencia AS Dependencia
FROM Dependencia AS dep;
GO
```

6. RESULTADO OBTENIDO

ID	Dependencia
1	Centro de Desarrollo de Software (CDS)
2	Dirección de Gestión y Finanzas
3	Talento y Desarrollo Humano
4	Logística y Abastecimiento
5	Asesoría Legal
6	Planificación y Presupuestos
7	Secretaría Administrativa
8	Operaciones y Servicios
9	Comunicación Institucional
10	Atención y Experiencia al Usuario

7. RESUMEN DEL PROCESO



8. CONCLUSIÓN

El uso estratégico de alias de columnas (AS) permite presentar la información de las dependencias de manera más clara, ordenada y profesional, facilitando el análisis y la toma de decisiones.

KEVIN ALFREDO MACHA BRUNO



¡ESTO APENAS COMIENZA!

```
-- Proyecto 6:
-- Enunciado: Sumar 4 GB adicionales a la memoria RAM mostrada

SELECT
    idEquipo AS [ID Equipo],
    RAM AS [RAM Actual],
    TRY_CAST(LEFT(RAM, LEN(RAM) - 2) AS INT) + 4 AS [RAM + 4GB]
FROM Hardware;
GO
```



ESTA BATALLA SERÁ ÉPICA.

ID Equipo	RAM Actual	RAM + 4GB
1 1	16GB	20
2 2	16GB	20
3 3	8GB	12
4 4	4GB	8
5 5	32GB	36
6 6	8GB	12
7 7	12GB	16
8 8	16GB	20
9 9	8GB	12
10 8	8GB	12

¡BIP BIP BIP!

PROYECTO 6: ANÁLISIS DE CONSULTA SQL

Base de Datos: **InventarioEquipos**

1. TABLA INVOLUCRADA Y DESCRIPCIÓN



Tabla: Hardware

Almacena los detalles técnicos y de componentes de los equipos de inventario. Contiene campos como ID Equipo, RAM Actual (Texto).

2. RELACIÓN ENTRE TABLAS

(Metodología Crows Foot)

Equipo
PK: idEquipo



Hardware
FK/PK: idEquipo

Cada registro de Hardware está asociado a un único Equipo.

3. ¿HAY INNER JOIN EN ESTA CONSULTA?

? JOIN NO

Es una selección simple de datos de una única tabla ("Hardware"). No se están combinando datos de múltiples tablas.

4. DIAGRAMA DE VENN

(Representación Conceptual)

Tabla Hardware
(Todos los Registros)

■ Todos los Registros de la tabla Hardware

🔍 Columnas Seleccionadas:
ID Equipo, RAM Actual, RAM + 4GB)
(con Alias)

5. CONSULTA SQL

```
-- Proyecto 6: Sumar 4 GB adicionales a la memoria RAM mostrada

SELECT
    idEquipo AS [ID Equipo],
    RAM AS [RAM Actual],
    TRY_CAST(LEFT(RAM, LEN(RAM) - 2) AS INT) + 4 AS [RAM + 4GB]
FROM Hardware;
GO
```

6. RESULTADO OBTENIDO

ID Equipo	RAM Actual	RAM + 4GB
1 1	16GB	20
2 2	16GB	20
3 8	8GB	12
4 4	4GB	8
5 6	32GB	36
6 6	8GB	12
7 7	12GB	16
8 8	16GB	20
9 9	8GB	12
10 8	8GB	12

7. RESUMEN DEL PROCESO






1. Selección de datos de la tabla Hardware
2. Elección de columnas clave: ID Equipo, RAM Actual
3. Aplicación de alias (AS) para renombrar columnas
4. Visualización del resultado con nuevos nombres

8. ¿QUÉ ES TRY_CAST, LEN Y AS INT?

TRY_CAST(value AS datatype)



Intenta convertir un valor. Si falla, devuelve NULL (Robusto).

LEN(string_expression)



Devuelve la longitud de una cadena (sin espacios finales).

AS INT

AS INT

Especifica que la conversión es a tipo de datos Entero.

9. CONCLUSIÓN



- ✓ Demuestra técnicas robustas para manipular datos de texto (RAM Actual), realizar cálculos numéricos seguros y presentar información clara y comprensible.
- ✓ TRY_CAST previene errores por datos no válidos, asegurando la integridad de los resultados comerciales.



```
-- Proyecto 7:
-- Enunciado: Calcular el doble de almacenamiento
-- =====
WITH CapacidadTotal AS (
    SELECT
        h.idEquipo,
        h.disco_cap,
        SUM(
            CASE
                WHEN part.unit = 'TB' THEN part.num * 1024.0
                ELSE part.num * 1.0
            END
        ) AS total_gb
    FROM Hardware h
    CROSS JOIN Equipos e
    SELECT TRIM(value) AS part_str
    FROM STRING_SPLIT(REPLACE(h.disco_cap, '+', '-'), '-')
) AS partes (
    SELECT
        TRY_CAST(LEFT(part_str, PATINDEX('%[0-9]%', part_str) - 1) AS INT) AS num,
        WHEN part_str LIKE '%TB%' THEN 'TB'
        WHEN part_str LIKE '%GB%' THEN 'GB'
        ELSE 'GB'
    ) AS num,
    CASE WHEN total_gb * 2 < 1024.0 AS DECIMAL(10,2)) AS VARCHAR) * ' TB'
FROM CapacidadTotal
SELECT * FROM CapacidadTotal
```

	idEquipo	Capacidad Actual	Doble de Capacidad
1	1	512GB SSD	1.00 TB
2	2	1TB SSD	2.00 TB
3	3	256GB SSD	512 GB
4	4	500GB HDD	1000 GB
5	5	512GB SSD	1.00 TB
6	7	1TB HDD	2.00 TB
8	7	528GB SSD	1.00 TB
9	8	256GB SSD	512 GB
9	9	512GB SSD	1.00 TB
10	10	1TB HDD	2.00 TB
11	11	512GB SSD	1.00 TB
12	12	256GB SSD	512 GB
14	13	512GB SSD	1.00 TB
18	16	1TB HDD	2.00 TB
18	12	512GB SSD	1.00 TB
18	17	1TB HDD	2.00 TB
18	18	512GB SSD	1.00 TB
18	19	256GB SSD	512 GB
21	20	500GB HDD	1000 GB
21	21	512GB SSD	1.00 TB
21	22	1TB HDD	2.00 TB
24	23	256GB SSD	512 GB
25	25	1TB HDD	2.00 TB
...	...	...	...



PROYECTO 7: CALCULAR EL DOBLE DE ALMACENAMIENTO



1. TABLA INVOLUCRADA Y DESCRIPCIÓN

Hardware (h)



Hardware

Descripción: Contiene información de hardware (equipos), incluyendo el campo 'disco_cap' que almacena la capacidad del disco duro.

2. RELACIÓN ENTRE TABLAS (Metodología Crows Foot)



Cada registro de Equipos está asociado a un único Hardware.

3. ¿HAY INNER JOIN EN ESTA CONSULTA?

YES / NO

Es una selección simple de datos de una única tabla (Hardware). No se están combinando datos de múltiples tablas.



4. DIAGRAMA DE VENN (Representación Conceptual)



Todos los Registros de la tabla Hardware

Columnas Seleccionadas: idEquipo, RAM Actual, RAM + 4GB

5. CONSULTA SQL

```
-- Proyecto 7: Calcular el doble de almacenamiento
-- =====
WITH CapacidadTotal AS (
    SELECT
        h.idEquipo,
        h.disco_cap,
        SUM(
            CASE
                WHEN part.unit = 'TB' THEN part.num * 1024.0
                ELSE part.num * 1.0
            END
        ) AS total_gb
    FROM Hardware h
    CROSS JOIN Equipos e
    SELECT TRIM(value) AS part_str
    FROM STRING_SPLIT(REPLACE(h.disco_cap, '+', '-'), '-')
) AS partes (
    SELECT
        TRY_CAST(LEFT(part_str, PATINDEX('%[0-9]%', part_str) - 1) AS INT) AS num,
        WHEN part_str LIKE '%TB%' THEN 'TB'
        WHEN part_str LIKE '%GB%' THEN 'GB'
        ELSE 'GB'
    ) AS num,
    CASE WHEN total_gb * 2 < 1024.0 AS DECIMAL(10,2)) AS VARCHAR) * ' TB'
FROM CapacidadTotal
SELECT * FROM CapacidadTotal
```

6. RESULTADO OBTENIDO

idEquipo	Capacidad Actual	Doble de Capacidad
1	512GB SSD	1.00 TB
2	1TB SSD	2.00 TB
3	256GB SSD	512 GB
4	500GB HDD	1000 GB
5	512GB SSD	1.00 TB
...	...	...
44	512GB SSD	1.00 TB
45	1TB HDD	2.00 TB



7. RESUMEN DEL PROCESO



- Selección de datos de la tabla Hardware
- Elección de columnas clave: ID Equipo, RAM Actual
- Aplicación de alias (AS) para renombrar columnas
- Visualización del resultado con nuevos nombres



8. ¿QUÉ ES TRY_CAST, LEN Y AS INT?

TRY_CAST

Intenta convertir un valor. Si falla, devuelve NULL (Robusto).

LEN

Devuelve la longitud de una cadena (sin espacios finales).

AS INT

Especifica que la conversión es a tipo de datos Entero.

9. CONCLUSIÓN



- Demuestra técnicas robustas para manipular datos de texto (RAM Actual), realizar cálculos numéricos exactos y presentar información clara y comprensible.
- TRY_CAST previene errores por datos no válidos, asegurando la integridad de los resultados comerciales.





-- Proyecto 8:
-- Enunciado: Demostrar cómo evitar errores de división
=====

-- Evita división por cero usando NULLIF
=====

```
SELECT 100 / NULLIF(0, 0) AS [Resultado Seguro];  
GO
```

Resultados	Mensajes	Estadísticas de clientes
Resultado Seguro		
1	NULL	

	idEquipo	RAM Actual	RAM + 4GB
1	1	512GB SSD	1.00 TB
2	2	1TB SSD	2.00 TB
3	3	256GB SSD	512 GB
4	4	500GB HDD	1000 GB
5	5	512GB SSD	1.00 TB
6	6	1TB HDD	2.00 TB
7	7	256GB SSD	512 GB
8	8	512GB SSD	1.00 TB
9	9	1TB HDD	2.00 TB
10	10	512GB SSD	1.00 TB
...	...	...	...



PROYECTO 8: CÓMO EVITAR ERRORES DE DIVISIÓN POR CERO USANDO NULLIF

1. TABLA INVOLUCRADA Y DESCRIPCIÓN

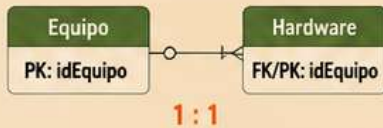


Tabla: **Hardware** (h)

Contiene información de hardware de los equipos, incluyendo el campo "disco_cap" que almacena la capacidad del disco duro.

2. RELACIÓN ENTRE TABLAS

(Metodología Crows Foot)



Cada registro de Equipo está asociado a un único Hardware.

3. ¿HAY INNER JOIN EN ESTA CONSULTA?

YES / NO

Es una selección simple de datos de una única tabla (Hardware). No se están combinando datos de múltiples tablas.



4. DIAGRAMA DE VENN (Representación Conceptual)



Todos los Registros de la tabla Hardware

Columnas Seleccionadas: idEquipo, RAM Actual, RAM + 4GB

5. CONSULTA SQL

-- Proyecto 8: Demostrar cómo evitar errores de división
-- Evita división por cero usando NULLIF
=====

```
SELECT 100 / NULLIF(0, 0) AS [Resultado Seguro];  
GO
```

6. RESULTADO OBTENIDO

Resultados	Mensajes	Estadísticas de clientes
Resultado Seguro		
1	NULL	

El resultado es NULL. La división por NULL produce NULL, no un error.

7. RESUMEN DEL PROCESO



8. ¿QUÉ ES NULLIF(0, 0)?

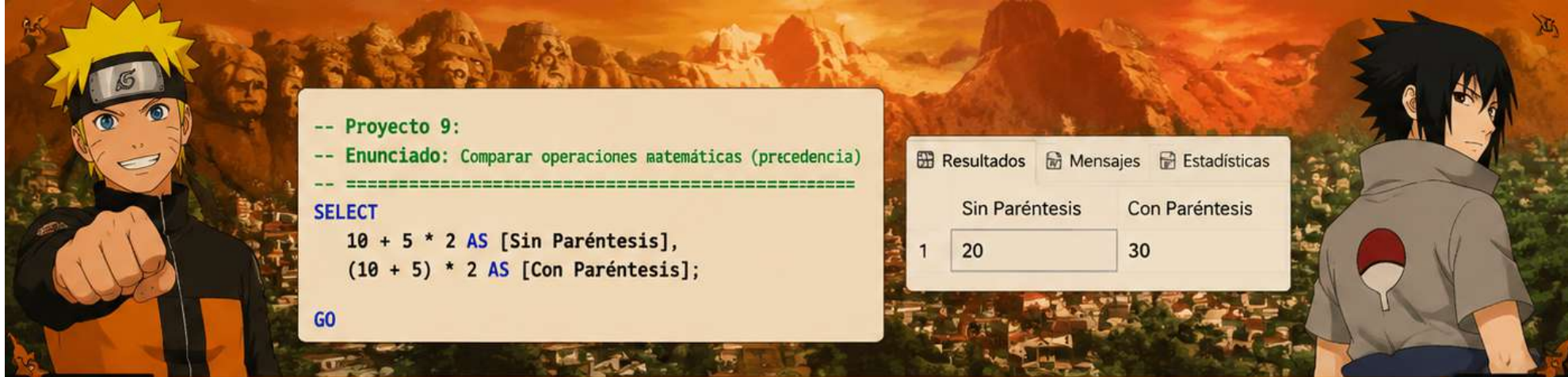


9. CONCLUSIÓN



- ✓ El uso de NULLIF protege contra errores de división por cero.
- ✓ Permite realizar cálculos seguros y mantener la estabilidad de las consultas.
- ✓ SQL Server maneja la división por NULL devolviendo NULL, evitando interrupciones en la ejecución.





-- Proyecto 9:
-- Enunciado: Comparar operaciones matemáticas (precedencia)
=====

SELECT

10 + 5 * 2 AS [Sin Paréntesis],
(10 + 5) * 2 AS [Con Paréntesis];

GO

Resultados Mensajes Estadísticas

Sin Paréntesis Con Paréntesis

	Sin Paréntesis	Con Paréntesis
1	20	30



PROYECTO 9: COMPARAR OPERACIONES MATEMÁTICAS Y PRECEDENCIA



1. CONSULTA SQL

-- Proyecto 9: Comparar operaciones matemáticas (precedencia)
-- =====

SELECT

10 + 5 * 2 AS [Sin Paréntesis],
(10 + 5) * 2 AS [Con Paréntesis];

GO

2. RESULTADO OBTENIDO

Resultados Mensajes Estadísticas

	Sin Paréntesis	Con Paréntesis
1	20	30

Total: 1 fila

3. RESUMEN DEL PROCESO

Proceso Sin Paréntesis

1. Multiplicación
(5*2=10)
2. Adición
(10+10=20)



10 + [5 * 2]
↓
10 + 10
↓
20

Proceso Con Paréntesis

1. Adición
(10+5=15)
2. Multiplicación
(15*2=30)



(10 + 5) * 2
↓
[15] * 2
↓
30



4. CONCLUSIÓN



El uso de **paréntesis** (paréntesis) **sobrescribe** la **precedencia** predeterminada de los operadores. Sin paréntesis, la **multiplicación** (*) tiene mayor prioridad que la **adición** (+). Los **paréntesis fuerzan** un orden específico, lo cual es crítico para cálculos precisos.





-- Proyecto 10:
-- Enunciado: Mostrar valores nulos reemplazados

```
SELECT
  idEquipo,
  observación AS [Observación Original],
  CASE
    WHEN observación IS NULL OR observación = '' THEN 'Sin observación'
    ELSE observación
  END AS [Observación Modificada],
  CASE
    WHEN observación IS NULL THEN 'Es NULL'
    WHEN observación = '' THEN 'Es cadena vacía'
    ELSE 'Tiene texto'
  END AS [Tipo de dato original]
FROM Equipo;
GO
```

idEquipo	Observación Original	Observación Modificada	Tipo de dato original
1	Equipo asignado a Dirección	Equipo asignado a Dirección	Tiene texto
2	Uso exclusivo OTI	Uso exclusivo OTI	Tiene texto
3	Sin observación	Sin observación	Es cadena vacía
4	Teclado con fallas	Teclado con fallas	Tiene texto
5	Asignado a asesoría jurídica	Asignado a asesoría jurídica	Tiene texto
6	Sin observación	Sin observación	Es cadena vacía
7	Batería con desgaste	Batería con desgaste	Tiene texto
8	Recepción	Recepción	Tiene texto
9	Gabinete oxidado	Gabinete oxidado	Tiene texto
10	Gerencia	Gerencia	Tiene texto
11	No enciende, en reparación	No enciende, en reparación	Tiene texto
12	Pantalla con líneas manchadas	Pantalla con líneas manchadas	Tiene texto
13	Imagen institucional	Imagen institucional	Tiene texto
14	Disco lento	Disco lento	Tiene texto
15	Sin observación	Sin observación	Es cadena vacía
16	Teclado con tecla Enter dañada	Teclado con tecla Enter dañada	Tiene texto
17	Equipo nuevo	Equipo nuevo	Tiene texto
18	No tiene RAM suficiente	No tiene RAM suficiente	Tiene texto
19	Gerencia de Operaciones	Gerencia de Operaciones	Tiene texto
20	Cámara no funciona	Cámara no funciona	Tiene texto



PROYECTO 10: LIMPIEZA DE DATOS DE INVENTARIO - INVENTARIOEQUIPOS



1. TABLA INVOLUCRADA Y DESCRIPCIÓN



Tabla fuente: 'Equipo'

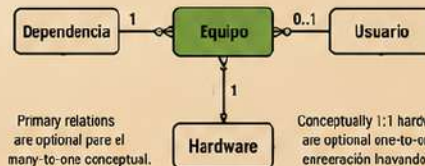
Contiene información general de los equipos de IT (tipo, estado, ubicación). Crucial para vincular hardware, partes y software.

Key columns:

- idEquipo
- observación, Observación
- tipo_equipo, partes software, etc.

2. RELACIÓN ENTRE TABLAS (Metodología Crows Foot)

Primary relations optional from la esumema



Primary relations are optional para el many-to-one conceptual.

Conceptually 1:1 hardware are optional one-to-one enreeración havando.

3. ¿HAY INNER JOIN EN ESTA CONSULTA?

~~INNER JOIN~~

No, no hay ningún INNER JOIN. La consulta combina información de la tabla 'Equipo' y no se combina información de múltiples tablas.



4. DIAGRAMA DE VENN (Representación Conceptual)



Todos los registros (N) son seleccionados. La lógica se aplica a nivel de columna para la observación.

5. CONSULTA SQL

```
-- Proyecto 10: Mostrar valores nulos reemplazados
SELECT
  idEquipo,
  observación AS [Observación Original],
  CASE
    WHEN observación IS NULL OR observación = '' THEN 'Sin observación'
    ELSE observación
  END AS [Observación Modificada],
  CASE
    WHEN observación IS NULL THEN 'Es NULL'
    WHEN observación = '' THEN 'Es cadena vacía'
    ELSE 'Tiene texto'
  END AS [Tipo de dato original]
FROM Equipo;
GO
```

NULL handling

alias

alias assignment

6. RESULTADO OBTENIDO

idEquipo	Observación Original	Observación Modificada	Tipo de dato original
1	Equipo asignado a Dirección	Equipo asignado a Dirección	Tiene texto
2	Uso exclusivo OTI	Uso exclusivo OTI	Tiene texto
3	Sin observación	Sin observación	Es cadena vacía
4	Teclado con fallas	Teclado con fallas	Tiene texto
5	Asignado a asesoría jurídica	Asignado a asesoría jurídica	Tiene texto
...	...	...	Tiene texto

Resultado organizado, limpio y clasificado.

7. RESUMEN DEL PROCESO



8. EN QUÉ MOMENTO USAR CASE Y WHEN

- Manejar valores NULL o vacíos de forma amigable.
- Clasificar datos según condiciones definidas.
- Reemplazar valores nulos o vacíos con texto descriptivo (e.g., 'Sin observación').
- Crear columnas calculadas dinámicas para mejor comprensión del dato.



9. CONCLUSIÓN

- ✓ La consulta mejora la calidad y comprensión de los datos en la base de datos InventarioEquipos.
- ✓ El uso de CASE y WHEN permite identificar y reemplazar valores nulos o vacíos.
- ✓ Esto facilita reportes más claros y decisiones basadas en datos confiables.



-- Proyecto 11:
-- Enunciado: Mostrar mensaje personalizado

```
SELECT 'Hola, ' + nombreUsuario AS [Mensaje]
FROM Usuario;
GO
```

Resultados Mensajes Estadísticas de clientes

Mensaje

1	Hola, Carlos Uzumaki
2	Hola, Sakura Haruno
3	Hola, Sasuke Uchiha
4	Hola, Kakashi Hatake
5	Hola, Might Guy
6	Hola, Shikamaru Nara
7	Hola, Ino Yamanaka
8	Hola, Choji Akimichi
9	Hola, Neji Hyuga
10	Hola, Rock Lee
11	Hola, Tenten
12	Hola, Kiba Inuzuka
13	Hola, Shino Aburame
14	Hola, Hinata Hyuga
15	Hola, Temari
16	Hola, Gaara
17	Hola, Jiraiya
18	Hola, Tsunade
19	Hola, Orochimaru
20	Hola, Itachi Uchiha
21	Hola, Deidara
22	Hola, Hidan
23	Hola, Pain (Yahiko)
24	Hola, Konan
25	Hola, Madara Uchiha
26	Hola, Obito Uchiha
27	Hola, Minato Namikaze
28	Hola, Hashirama Senju

PROYECTO 11: VISUALIZACIÓN DE LAPTOPS EN BUEN ESTADO - INVENTARIOEQUIPOS

1. TABLA INVOLUCRADA Y DESCRIPCIÓN



Tabla fuente: Equipo
Contiene la información general de todos los equipos, incluyendo tipo y estado. Crucial para vincular hardware, partes y software.

Key columns incluyen:

- idEquipo, tipo_equipo, estado, dependencia, etc.

2. RELACIÓN ENTRE TABLAS (Metodología crows foot)



3. ¿HAY INNER JOIN EN ESTA CONSULTA?

NO

INNER JOIN

Esta es una selección simple de una única tabla ('Equipo') con condiciones, por lo que **NO** se combina información de múltiples tablas.

4. DIAGRAMA DE VENN (Representación Conceptual)



5. CONSULTA SQL

```
-- Proyecto 11: Mostrar todos los laptops cuyo estado es BUENO
SELECT *
FROM Equipo
WHERE tipo_equipo = 'LAPTOP' AND estado = 'BUENO';
GO
```

conceptual con projected select Image (*)

conditions (condiciones)

6. RESULTADO OBTENIDO

idEquipo	tipo_equipo	estado	dependencia	estado
2	LAPTOP	BUENO	...	10
4	LAPTOP	BUENO	...	13
15	LAPTOP	BUENO	...	13
24	LAPTOP	BUENO	...	13
28	LAPTOP	BUENO	...	...

5 filas encontradas...

7. RESUMEN DEL PROCESO

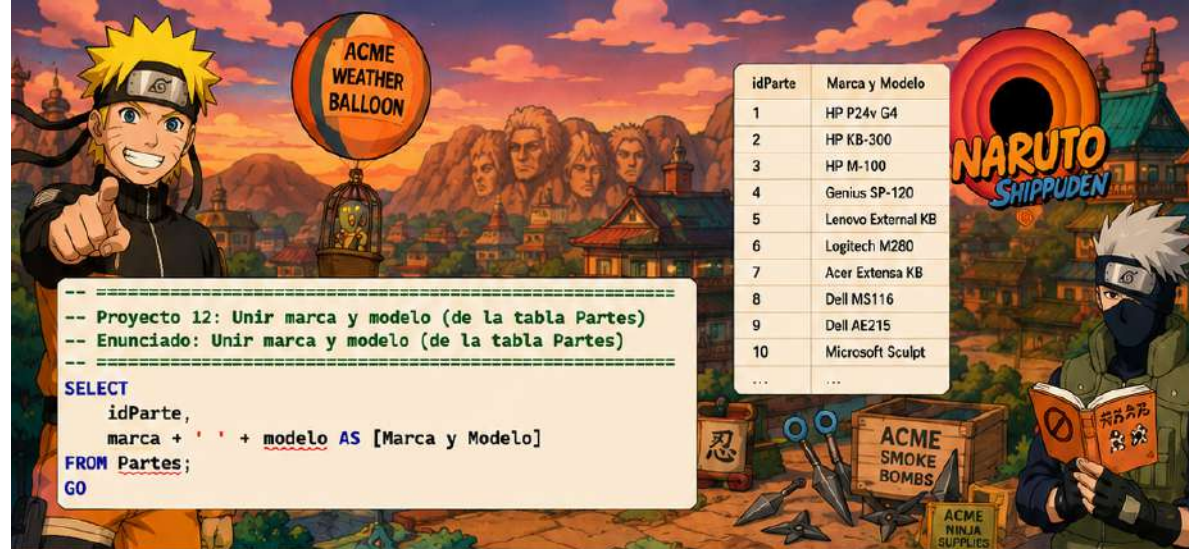


8. EN QUÉ MOMENTO USAR CASE Y WHEN

- Estructura Condicional SQL
- Para transformar valores basados en condiciones.
- Clasificar o reemplazar datos (ej.: NULL, vacío, texto).
- Crear columnas calculadas dinámicas para mejorar la comprensión del dato.

9. CONCLUSIÓN

- La consulta demuestra técnicas efectivas para filtrar y visualizar datos en la base de datos InventarioEquipos.
- El uso de filtros WHERE optimiza la recuperación de información relevante.
- Estos resultados facilitan decisiones rápidas y precisas en la gestión de inventarios.



ACME
WEATHER
BALLOON

idParte	Marca y Modelo
1	HP P24v G4
2	HP KB-300
3	HP M-100
4	Genius SP-120
5	Lenovo External KB
6	Logitech M280
7	Acer Extensa KB
8	Dell MS116
9	Dell AE215
10	Microsoft Sculpt
...	...

-- Proyecto 12: Unir marca y modelo (de la tabla Partes)

-- Enunciado: Unir marca y modelo (de la tabla Partes)

```

SELECT
  idParte,
  marca + ' ' + modelo AS [Marca y Modelo]
FROM Partes;
GO
    
```

NARUTO
SHIPPUDEN

ACME
SMOKE
BOMBS

ACME
NINJA
SUPPLIES

PROYECTO 12: UNIR MARCA Y MODELO (DE LA TABLA PARTES) ✨

1. TABLA INVOLUCRADA Y DESCRIPCIÓN

TABLA: Partes

Almacena información detallada sobre los componentes de hardware de cada equipo.

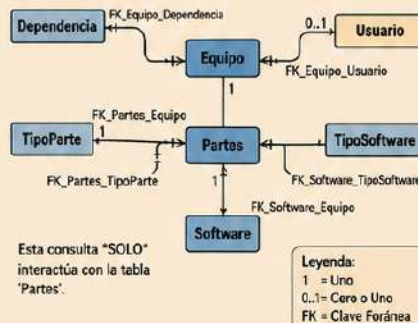
Columnas clave:

- idParte (FK)
- idEquipo (FK)
- idTipoParte (FK)
- marca
- modelo
- nro_serie

Columnas	Tipo
idParte	(FK, INT)
idEquipo	(FK, INT)
idTipoParte	(FK, INT)
marca	VARCHAR(100)
modelo	VARCHAR(100)
nro_serie	VARCHAR(100)

marca
+
modelo

2. RELACIÓN ENTRE TABLAS (CROW'S FOOT)



3. ¿HAY INNER JOIN EN ESTA CONSULTA?



INNER JOIN

No, no hay INNER JOIN. La consulta solo accede a una única tabla (Partes).

4. CONSULTA SQL

```

-- Proyecto 12: Unir marca y modelo
-- (de la tabla Partes)

SELECT
  idParte,
  marca + ' ' + modelo AS [Marca y Modelo]
FROM Partes;
GO
    
```

5. RESULTADO OBTENIDO

idParte	Marca y Modelo
1	HP P24v G4
2	HP KB-300
3	HP M-100
4	Genius SP-120
5	Lenovo External KB
...	...

Muestra 10 filas total (ejemplo de 5)

6. RESUMEN DEL PROCESO



7. EN QUÉ MOMENTO USAR CASE Y WHEN

- ✦ Estructura Condicional SQL
- ✦ Para transformar valores basados en condiciones.
- ✦ Clasificar o reemplazar datos (ej.: NULL, vacío, texto).
- ✦ Crear columnas calculadas dinámicas para mejor comprensión del dato.



8. CONCLUSIÓN

- ✓ Esta consulta simple demuestra el uso efectivo de operadores de manipulación de cadenas en SQL (concatenación) para crear un campo de reporte más legible y amigable para el usuario.
- ✓ El resultado facilita la identificación rápida de componentes mediante la combinación de marca y modelo en una sola columna.





-- Proyecto 13: Mostrar información formateada (con espacios)
-- Enunciado: Mostrar información formateada (con espacios)

SELECT

```
Equipo + ' - ' + CAST(idEquipo AS VARCHAR) + ' - Tipo: ' +
tipo_equipo AS [Información Formateada]
FROM Equipo;
GO
```

Información Formateada	
1	Equipo 1 - Tipo: ESCRITORIO
2	Equipo 2 - Tipo: LAPTOP
3	Equipo 3 - Tipo: ALL IN ONE
4	Equipo 4 - Tipo: ESCRITORIO
5	Equipo 5 - Tipo: LAPTOP
6	Equipo 6 - Tipo: ESCRITORIO
7	Equipo 7 - Tipo: ESCRITORIO
8	Equipo 8 - Tipo: LAPTOP
9	Equipo 9 - Tipo: ALL IN ONE
10	Equipo 10 - Tipo: ESCRITORIO
11	Equipo 11 - Tipo: ESCRITORIO
12	Equipo 12 - Tipo: LAPTOP
13	Equipo 13 - Tipo: ALL IN ONE
14	Equipo 14 - Tipo: ESCRITORIO
15	Equipo 15 - Tipo: LAPTOP
16	Equipo 16 - Tipo: ESCRITORIO
17	Equipo 17 - Tipo: ESCRITORIO
18	Equipo 18 - Tipo: LAPTOP
19	Equipo 19 - Tipo: ALL IN ONE
20	Equipo 20 - Tipo: ESCRITORIO



ANÁLISIS DE CONSULTA SQL: PROYECTO 13 (MOSTRAR INFORMACIÓN FORMATEADA)

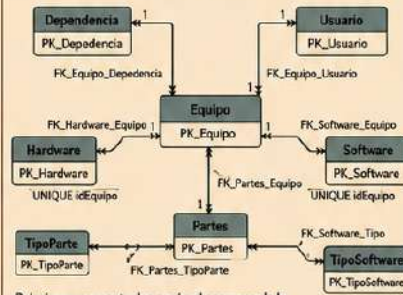
1. TABLA INVOLUCRADA Y DESCRIPCIÓN



Tabla: Equipo
Almacena la información principal de los equipos de inventario, incluyendo su identificador y tipo.

Columna	Tipo
idEquipo	INT (PK)
tipo_equipo	VARCHAR(50)
Equipo	VARCHAR(100)

2. RELACIÓN ENTRE TABLAS (Crow's Foot)



Relaciones conceptuales según el esquema de la base de datos "InventarioEquipos".

3. ¿HAY INNER JOIN EN ESTA CONSULTA?

NO

La consulta no realiza ningún operador JOIN. Solo accede a una única tabla.



4. DIAGRAMA DE VENN (Representación Conceptual)



No aplica combinación con otras tablas.

5. CONSULTA SQL

```
-- Proyecto 13: Mostrar información formateada (con espacios)
SELECT
Equipo + ' - ' + CAST(idEquipo AS VARCHAR) +
' - Tipo: ' + tipo_equipo AS [Información Formateada]
FROM Equipo;
GO
```

6. RESULTADO OBTENIDO

Información Formateada	
1	Equipo 1 - Tipo: ESCRITORIO
2	Equipo 2 - Tipo: LAPTOP
3	Equipo 3 - Tipo: ALL IN ONE
4	Equipo 4 - Tipo: ESCRITORIO
5	Equipo 5 - Tipo: LAPTOP
...	...

Genera 20 filas total (ejemplo de 20)



7. RESUMEN DEL PROCESO



8. ¿EN QUÉ MOMENTO USAR CASE Y WHEN?



- ★ Estructura Condicional SQL
- ★ Para transformar valores basados en condiciones.
- ★ Clasificar o reemplazar datos (ej: NULL, vacío, texto).
- ★ Crear columnas calculadas dinámicas para mejor comprensión del dato.

9. CONCLUSIÓN

- ✓ Esta consulta simple demuestra el uso efectivo de funciones de conversión (CAST) y operadores de concatenación (+) para formatear datos directamente en SQL.
- ✓ Permite generar reportes más legibles y amigables para el usuario, presentando información combinada en un solo campo.
- ✓ Es una técnica útil para auditorías rápidas, vistas personalizadas e informes sin necesidad de herramientas externas.







火影



-- Proyecto 14: Concatenación con valores nulos (evitar problemas)

-- Enunciado: Concatenar nombre de usuario con rol (Comprador)

-- omitiendo valores NULL

```

SELECT
CONCAT(COALESCE(combreUsuario, ''),
CASE
WHEN COALESCE(combreUsuario, '') = '' THEN ''
ELSE ' - '
END,
COALESCE(rol, 'Comprador')) AS [Concatenación Segura]
FROM Usuario;

```

6. RESULTADO OBTENIDO

idUsuario	Concatenación Segura
1	Carlos Augusto Gonzales Quispe - Comprador
2	Maria Esperanza Benites Flores - Comprador
3	Jorge Luis Huamán Mendoza - Comprador
4	Ana Lucia Ramirez Tello - Comprador
5	Luis Alberto Villanueva Rojas - Comprador
6	Miguel Angel Quispe Aquino - Comprador
7	Sonia Marisol Cárdenas Uribe - Comprador
8	Walter Jesús Neri Salas Luna - Comprador
9	Elizabeth Paola Mendoza Soas - Comprador
10	Raúl Iván Velásquez Ortiz Vega - Comprador
11	David Venancio Chipo Huayta - Comprador
12	Oscar Raúl Condon Torres - Comprador
13	Hector Pablo Cullar Medina - Comprador
14	Rosmary Isabella Castro León - Comprador
15	Freddy Román Huamán Yana - Comprador
16	Alejandro Marín Ayquipa Beltrán - Comprador
17	Nancy Beatriz López Alvarado - Comprador
18	Javier Arturo Quispe Mamani - Comprador
19	...

INFOGRAFÍA: PROYECTO 14 - CONCATENACIÓN SEGURA

ANÁLISIS Y RESULTADOS EN SQL SERVER

1. TABLA INVOLUCRADA Y DESCRIPCIÓN

Tabla: Usuario
Almacena información de los usuarios del sistema, incluyendo su nombre de usuario y rol.

Columna	Tipo	Descripción
idUsuario	INT	Identificador único
nombreUsuario	VARCHAR(100)	Nombre del usuario
rol	VARCHAR(50)	Rol del usuario

2. RELACIÓN ENTRE TABLAS (Crow's Foot)

Usuario
PK idUsuario
nombreUsuario
rol

1 — ∞

Equipo
PK idEquipo
FK idUsuario
tipo_equipo

Un **Usuario** puede tener uno o muchos **Equipos**.

3. ¿HAY INNER JOIN EN ESTA CONSULTA?



NO

La consulta no realiza ningún **INNER JOIN**. Solo accede a la tabla **Usuario**.

4. DIAGRAMA DE VENN (Manejo de NULLs)



CONCAT con COALESCE evita resultados NULL

5. CONSULTA SQL

```

-- Proyecto 14: Concatenación con valores nulos (evitar problemas)
-- Concatenar nombre de usuario con rol (Comprador)
SELECT
CONCAT(
COALESCE(combreUsuario, ''),
CASE
WHEN COALESCE(combreUsuario, '') = '' THEN ''
ELSE ' - '
END,
COALESCE(rol, 'Comprador')
) AS [Concatenación Segura]
FROM Usuario;

```

6. RESULTADO OBTENIDO

idUsuario	Concatenación Segura
1	Carlos Augusto Gonzales Quispe - Comprador
2	Maria Esperanza Benites Flores - Comprador
3	Jorge Luis Huamán Mendoza - Comprador
4	Ana Lucia Ramirez Tello - Comprador
5	Luis Alberto Villanueva Rojas - Comprador
6	Miguel Angel Quispe Aquino - Comprador
7	Sonia Marisol Cárdenas Uribe - Comprador
8	Walter Jesús Neri Salas Luna - Comprador
9	Elizabeth Paola Mendoza Soas - Comprador
10	Raúl Iván Velásquez Ortiz Vega - Comprador
...	...

7. RESUMEN DEL PROCESO






- Leer datos de la tabla **Usuario**.
- Aplicar **COALESCE** para convertir NULL en "" o texto por defecto.
- CONCATENAR** nombreUsuario, separador " - " y rol (o "Comprador").
- Generar resultado concatenado de forma segura.

8. ¿PARA QUÉ SIRVE CONCAT?



- Unir cadenas de texto.
- Combinar múltiples columnas.
- Evitar NULL en el resultado final.
- Crear reportes más legibles.

9. CONCLUSIÓN

- ✓ La función **CONCAT** con **COALESCE** permite concatenar valores de forma segura, evitando que los **NULL** generen **NULL** en el resultado final.
- ✓ Mejora la calidad y legibilidad de los datos.
- ✓ Es una práctica recomendada en SQL Server para reportes y consultas dinámicas.



```

-- Proyecto 15: Uso de COALESCE y NULLIF
-- Enunciado: Mostrar valores alternativos
-- omitiendo valores NULL

SELECT
    idEquipo,
    COALESCE(NULLIF(observacion, ''), 'N/A') AS [Observación]
FROM Equipo;
GO
        
```

idEquipo	Observación
1	Equipo asignado a Dirección
2	Uso exclusivo OTI
3	NA
4	Teclado con fallas
5	Asignado a asesoría jurídica
6	NA
7	Batería con desgaste
8	Recepción
9	Gabinete oxidado
10	Gerencia
11	No enciende, en reparación
12	Pantalla con ligeras manchas
13	Imagen institucional
14	Disco lento
15	NA
16	Teclado con tecla Enter dañada
17	Equipo nuevo
18	No tiene RAM suficiente
19	Gerencia de Operaciones
20	Cámara no funciona





PROYECTO 15: USO DE COALESCE Y NULLIF

ANÁLISIS DE DATOS EN SQL SERVER



1. TABLA INVOLUCRADA Y DESCRIPCIÓN



Tabla: **Equipo**

Almacena los datos principales de cada equipo de inventario, incluyendo código, fechas, tipo, estado y observaciones.

Columna	Tipo	Descripción
idEquipo	INT	Identificador único
codigo	VARCHAR(20)	Código del equipo
observacion	VARCHAR(300)	Observaciones del equipo
estado	VARCHAR(20)	Estado actual del equipo

2. RELACIÓN ENTRE TABLAS (Metodología Crow's Foot)



Un Equipo pertenece a una Dependencia y un TipoEquipo, puede tener un Usuario asignado y usarse en un Hardware específico.

3. ¿HAY INNER JOIN EN ESTA CONSULTA?

NO
INNER JOIN

La consulta es un simple SELECT sobre una única tabla: Equipo.



4. DIAGRAMA DE VENN (Manejo de NULLs)



💡 `NULLIF(observacion, '')` convierte cadenas vacías en NULL para que `COALESCE` pueda reemplazarlas.

5. CONSULTA SQL

```

-- Proyecto 15: Uso de COALESCE y NULLIF
-- Mostrar valores alternativos omitiendo NULLs

SELECT
    idEquipo,
    COALESCE(NULLIF(observacion, ''), 'N/A') AS [Observación]
FROM Equipo;
GO
        
```

¿Cómo funciona?

- ✓ `NULLIF(observacion, '')` convierte cadenas vacías en NULL.
- ✓ `COALESCE()` reemplaza NULL por "N/A".

6. RESULTADO OBTENIDO

idEquipo	Observación (Concatenación Segura)
1	Equipo asignado a Dirección
2	Uso exclusivo OTI
3	N/A
4	Teclado con fallas
5	Asignado a asesoría jurídica
6	N/A
7	Batería con desgaste
8	Recepción
9	Gabinete oxidado
10	Gerencia
...	...

✓ Los NULL o cadenas vacías fueron reemplazados por "N/A".

7. RESUMEN DEL PROCESO



8. ¿PARA QUÉ SIRVE COALESCE?

- Permite mostrar valores alternativos si un campo es NULL.
- Mejora la presentación de reportes.
- Evita datos vacíos o sin información visible.



9. CONCLUSIÓN

- ✓ Usar `COALESCE` y `NULLIF` mejora la calidad de los datos mostrados, evitando valores NULL o vacíos.
- ✓ Facilita la lectura de informes y reportes en SQL Server.
- ✓ Es una práctica recomendada para consultas más limpias y profesionales.

